

AI-RedEye Harness — Design, Architecture & Improvements over VVAH

Document type: Technical design + executive briefing **Subject:** redeye — an agentic SAST harness for autonomous vulnerability discovery **Baseline of comparison:** Visa Vulnerability Agentic Harness (VVAH), Apache-2.0, the upstream project RedEye derives from

Executive summary — read this first

RedEye is an agentic, LLM-driven static analysis harness that finds security vulnerabilities in source code and delivers them as triage-ready, GitHub-native findings. It started from Visa's open-source **VVAH** and was substantially reworked to fix the three failure modes that keep AI-SAST out of production.

Why leadership should care — the four headline wins:

1. Fewer false positives. RedEye stacks *six* false-positive filters (vs VVAH's two), including a deterministic **K-of-N outcome verifier** that works even on models VVAH's voting cannot use. Net effect: reviewers spend their time on real bugs, not noise.

2. Less hallucination. RedEye adds a **7-layer hallucination-control stack**. The decisive addition VVAH lacks is a **deterministic grounding pass** that opens every cited file and rejects findings pointing at code that does not exist. The model can no longer invent file paths, line numbers, or sinks.

3. Lower cost. RedEye runs **zero-LLM deterministic filters first**, so the expensive model stages never burn tokens on findings that already failed a free check. Add an **incremental cache**, **diff-only PR scanning**, and a **zero-cost offline mode**, and total spend per scan drops materially versus a naive verify-everything loop.

4. Production-ready from day one. Unlike VVAH, RedEye ships native **CI/CD** (GitHub Actions), **PR comments with TP/FP checkboxes**, a **feedback loop** that learns from reviewer verdicts, **five more LLM backends**, and an **immutable audit trail** per run.

One-line positioning: *the same tool runs as a researcher's deep-dive on Monday and as a PR gate on Tuesday — and what reaches a human cites real code and survived every cheap check the harness can run.*

At-a-glance: RedEye vs VVAH

Dimension	VVAH (baseline)	RedEye
Pipeline stages	9	14 (5 fully deterministic)

False-positive filters	2 (prefilter + adversarial verify)	6 (policy gate, voting, validator, dedupe, outcome verifier , baseline)
Hallucination control	Evidence gate + adversarial verify	7 layers incl. structural pre-index + deterministic file:line grounding + PoC gate
Cited-path verification	Not verified deterministically	Opens the file, checks line + CWE tokens
FP control on no-temperature models (CLI/Opus)	Voting is a no-op	Deterministic K-of-N verifier still works
LLM backends	3 (cli, sdk, openai)	8 (+ Bedrock, Vertex, Ollama, mock)
Offline / zero-cost mode	—	Deterministic mock backend
PR diff scanning	Not built-in	--diff-only first-class
CI/CD integration	Implied via SARIF	Drop-in GitHub Actions workflow
Reviewer feedback loop	None	SQLite store, fed into next scan
Repeat-FP suppression	None	Baseline accept file
Incremental cost cache	None	Per-file structural cache
Audit trail	run manifest	run_manifest.json + hallucination metrics

Authorized use only. Findings are LLM-generated triage candidates that require human review. Run only against code you own or are authorized to test.

1. Purpose & scope

This document describes the **structure and design** of the AI-RedEye harness and then details, point by point, the **improvements over VVAH** — with particular emphasis on false positives, hallucination, and cost. It is written to serve both an executive skim (Section 0 above) and an engineering deep-dive (Sections 2–7).

RedEye is a derivative work of **VVAH** (Copyright Visa, Inc., Apache-2.0). The architecture and scaffolding derive from VVAH; the source has been substantially reworked and extended. Attribution is preserved in [NOTICE](#).

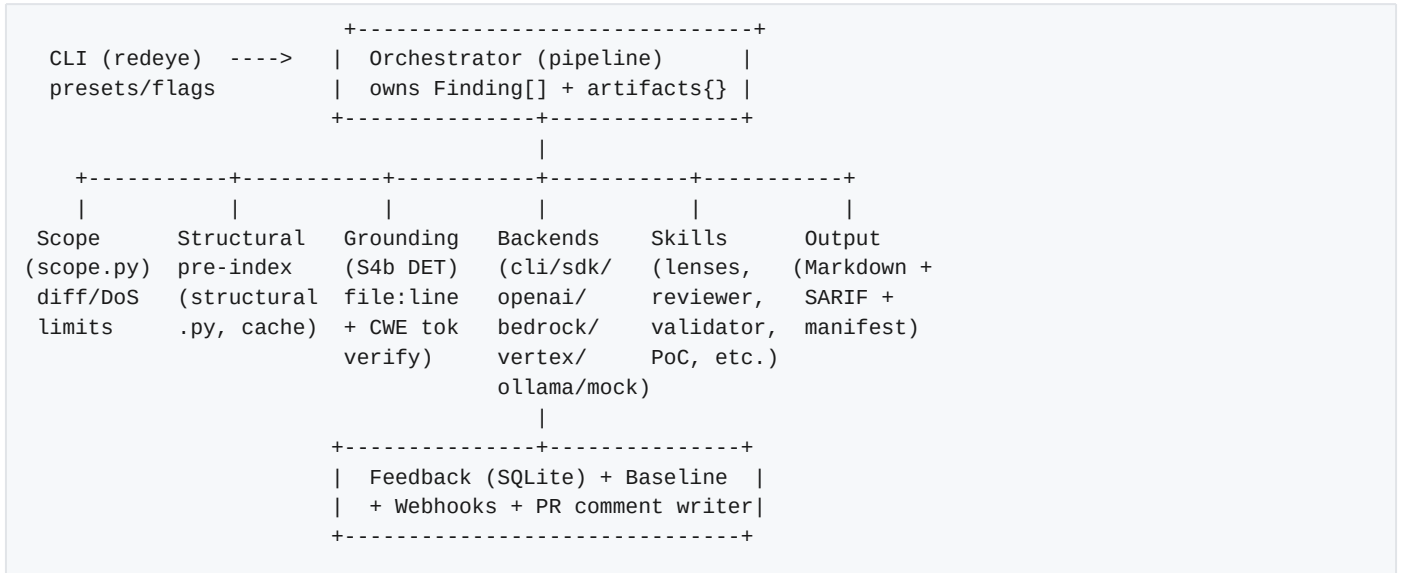
2. System design overview

2.1 Design principles

1. **Threat-model before analysis** — focus the attack surface so research lenses don't waste budget on low-impact areas.

2. **Deterministic checks carry the precision burden** — let the LLM be aggressive about *discovery*; let cheap, repeatable code decide what is *real*. Every deterministic layer is one the model cannot lie its way past.
3. **Cheap-first ordering** — free filters (file I/O, regex) run before any paid model call, so tokens are never spent confirming garbage.
4. **Dual-mode, one tool** — research deep-dive and PR gate are the same engine with different presets.
5. **Provider-agnostic** — no single LLM vendor is a hard dependency.
6. **Auditable by default** — every run is reproducible at the manifest level.

2.2 Component map



Key modules (in `redeye/`): `cli.py` (entry + presets), `scope.py` (file selection, diff, DoS caps), `structural.py` (deterministic pre-index), `grounding.py` (file:line + CWE-token verification), `cache.py` (incremental structural cache), `baseline.py` (accepted-finding fingerprints), `schema.py` (Finding / taint data model), plus `backends/`, `skills/`, `pipeline/`, `feedback/`, `notify/`, `output/`, `enterprise/`.

2.3 Data model (the Finding)

Every finding carries a structured **taint shape** — not just prose:

- `source`, `source_location` (file:line) — where attacker data enters
- `sink`, `sink_location` (file:line) — the dangerous operation
- `sanitizer_missing` (bool) and `sanitizers_observed` (list)
- `taint_path` — ordered file:line steps from source to sink
- `cwe`, `cvss_vector`, `cvss_score`, `severity`, `confidence`
- `tags` (e.g. `grounded:true`, `hallucinated:bad-path`, `weak-evidence`)
- `evidence[]` — PASS/FAIL rows from each deterministic check

A response without a source + sink is rejected at parse time — the model must hold a *theory* of the bug.

3. Pipeline structure (14 stages, 3 phases)

Five stages are **fully deterministic** (no LLM): S1b, S4b, S5, S7, and the voting tally in S6 — plus the deterministic S8c outcome verifier.

P Stage	Type	Role / purpose
A — Discovery & S1 attack_surface_mapper Modeling	LLM	Map HTTP/RPC endpoints, surfaces
S1b structural_index	DET	Regex/AST inventory of real routes/sources/sinks/secrets
S2 threat_modeler	LLM	STRIDE threat model
S3 research_strategist	LLM	Build the hunting plan
B — Deep Discovery & S4 research_lenses Verification	LLM	Language / crypto / logic / access-control / IaC lenses
S4b grounding_pass	DET	Verify cited file:line exists + CWE-family tokens nearby
S5 policy_gate	DET	Drop test/vendor paths, no-remediation, below severity floor
S6 adversarial_reviewer + voting	LLM (+DET tally)	Rewrite chain; N-of-M quorum vote
S6.5 validator	LLM (cheap)	Single-pass TP/FP gate; auto-rejects S4b failures
C — Synthesis, Chainage & S7 triage Reporting	DET	Merge same-root-cause findings
S8 exploit_strategist	LLM	Exploit-chain construction
S8b poc_gate	LLM (cheap)	Demand a concrete PoC; demote/drop if none
S8c outcome_verifier	DET	K-of-N verdict over 5 upstream signals

S9 emit	DET	Write Markdown + SARIF 2.1.0 + manifest
---------	-----	---

Stage contract: each stage is `run(ctx) -> StageResult`. The orchestrator owns the running `Finding[]` and `artifacts{}`; stages read from `ctx` and return what to store. A crashing stage sets `StageResult.error` and the pipeline continues; a backend error falls back to the deterministic mock so the run still completes; running out of budget stops the backend call but keeps partial work.

4. Improvement: fewer false positives (vs VVAH)

VVAH baseline: two FP defenses — `S5 Prefilter` (drops low-confidence / missing-evidence findings) and `S6 Verify` (adversarial TP/FP verdict + CVSS). Multi-agent voting exists in `S4` but **only works on temperature-capable backends** (`sdk/openai`); on the `claude CLI` or `Opus` it is a no-op.

RedEye adds four more independent FP filters and fixes the no-temperature gap:

- **Multi-agent voting (S6), run on the *refined* chain.** Voting happens after the adversarial reviewer rewrites the finding, so voters score the improved version. Cross-vendor voters (Anthropic + OpenAI + local) are recommended, because identical models make identical mistakes.
- **Single-pass validator (S6.5).** A cheap, dedicated TP/FP gate distinct from voting; receives a dossier of deterministic evidence + taint shape so it cannot rationalise away a hallucination.
- **Deterministic dedupe (S7).** One root cause described by five lenses is reported once.
- **Outcome verification (S8c) — the key differentiator.** A **temperature-free, deterministic K-of-N verdict** over five signals already gathered upstream: grounding, taint completeness, concrete PoC, reachability, and voter/validator agreement. Because it needs no sampling, **it suppresses false positives even on CLI/Opus models where VVAH's voting cannot run.**
- **Baseline acceptance.** Reviewed-and-accepted findings are fingerprinted (`sha256(cwe+path+line+skill)`) and filtered from every future report — robust to LLM rewording. VVAH has no equivalent.
- **Feedback loop.** Reviewer TP/FP marks persist to SQLite and are injected as calibration into the next scan's prompts.

Illustrative: the bundled deterministic demo filters 4 candidate findings down to **0 false positives** delivered, exercising the full stack end-to-end.

5. Improvement: less hallucination (vs VVAH)

VVAH baseline: relies on an *evidence gate* (`require_evidence` drops findings missing `source_ref/sink_ref`) plus adversarial verification. It does **not** deterministically confirm that a cited file/line actually exists, and it has **no structural pre-index** to anchor the model to real code.

RedEye's 7-layer hallucination-control stack:

Layer	Stage	What it kills	LLM cost	In VVAH?
-------	-------	---------------	----------	----------

Structural pre-index	S1b	Invented routes/sources/sinks/secrets	Zero	No
Taint-flow schema	S4	Findings with no source→sink theory	Zero	Partial (evidence refs)
Grounding pass	S4b	Cited file/line that doesn't exist; no CWE tokens	Zero	No
Validator auto-reject	S6.5	Findings already tagged hallucinated	Zero	No
PoC gate	S8b	Findings with no concrete exploit payload	Cheap	No
Multi-agent voting	S6	Findings only one voter backs	LLM×N	Partial (temp-only)
Outcome verifier	S8c	Findings failing K-of-N	Zero	No

The decisive additions VVAH lacks:

1. **Structural pre-index (S1b).** Before any LLM call, regex + AST extract a deterministic inventory of the target's *real* routes, sources, sinks, and secrets. Lenses reason "given these real sinks, which combos are dangerous?" instead of "imagine where bugs might be." The model can no longer invent code the repo doesn't contain.
2. **Grounding pass (S4b).** For each finding it (a) resolves the cited path (rejecting `..` escapes / nonexistent files), (b) checks the line is within the file, and (c) reads a ±5-line window for CWE-family tokens (CWE-89 → `execute/query/cursor`; CWE-78 → `subprocess/shell/os.system`). Outcomes: pass → keep; bad path/line → tag `hallucinated:*` (validator auto-rejects, or drop with `--strict-grounding`); missing tokens → `weak-evidence`, cap confidence ≤ 0.5. **This is the single biggest anti-hallucination lever and VVAH has nothing equivalent.**
3. **PoC gate (S8b).** The model must write a concrete payload (quoted string, URL, HTTP verb, injection metachar, `../`, `{`, `<script`, SQL keyword); placeholders are rejected. No payload → demote a severity notch (or drop with `--require-poc`). If the model can't write an exploit, it probably doesn't understand the bug.

Visibility: every run emits `hallucination_metrics` (`raw_lens`, `ungrounded_dropped`, `ungrounded_downgraded`, `validator_rejected`, `voted_out`, `missing_poc`, `outcome_unverified`) plus a per-finding PASS/FAIL evidence trail in the report — so operators see exactly how much noise was pruned.

6. Improvement: reduced cost (vs VVAH)

VVAH baseline: per-stage `max_budget_usd`, `max_tokens` caps, an estimate dry-run, and token accounting. Good controls — but it spends model tokens on the verify stage for findings that a free deterministic check could have killed, and it has no incremental cache and no diff-only scanning.

RedEye keeps all of VVAH's knobs and adds structural cost savings:

- **Cheap-first filter ordering.** Zero-LLM layers (S1b, S4b, S5, S7, S8c) run *before* the paid stages, so a finding citing a nonexistent path is dropped by file I/O before any model adversarially reviews, validates, or writes a PoC for it. This is the biggest structural saving versus VVAH's verify-then-filter flow.
 - **Validator auto-reject.** S6.5 skips the LLM call entirely for findings already tagged hallucinated — no tokens spent on the least-worthy findings.
 - **Incremental structural cache.** The pre-index is cached per file by (path, size, mtime_ns); unchanged files are served from cache on the next scan, only changed files re-scanned. (redeye/cache.py.)
 - **Diff-only PR scanning.** `--diff-only --pr-base origin/main` scopes a PR scan to just the changed files via `git diff` — VVAH has no first-class diff mode, so PR scans there re-analyse far more code.
 - **Scope / DoS limits.** `--max-files`, `--max-file-bytes`, `--max-total-bytes`, `--exclude-path`, default ignore dirs, and an interesting-extension allowlist keep irrelevant bytes out of every prompt.
 - **Cheap models where cheap suffices.** Validator and PoC gate are explicitly Haiku-/Flash-class; premium models are reserved for lenses and adversarial review via per-stage role→backend mapping.
 - **Zero-cost offline mode.** `--preset quick / make demo` runs the full 14-stage pipeline deterministically with **no API keys and no network** — ideal for CI smoke tests at literally zero LLM cost.
-

7. Other improvements (vs VVAH)

- **Dual-mode presets.** `quick / pr / ci / deep` collapse common flag sets into one switch; explicit flags always win. VVAH has no preset layer.
- **Five more backends.** Adds **AWS Bedrock**, **Google Vertex (Gemini)**, **Ollama (local)**, and the deterministic **mock** to VVAH's cli/sdk/openai — with automatic fallback to mock on backend error so runs always complete.
- **Native CI/CD.** Drop-in GitHub Actions workflow runs PR scan + full scan + feedback collection. SARIF 2.1.0 emits taint `codeFlows`, so GitHub Code Scanning renders inline source→sink traces.
- **PR comments + feedback loop.** Markdown comments carry `[ ] True Positive / [ ] False Positive` checkboxes; `collect-feedback` parses ticked boxes back into the SQLite store. VVAH has no formal feedback loop.
- **Webhooks.** Slack / Teams / Discord / generic, with optional HMAC signing.
- **Baseline file** (Section 4) — team-shareable accepted-finding decisions committed to source control.
- **First-run UX.** `redeye init` detects credentials and writes a tailored `.env`; `redeye doctor` reports exactly what's missing per profile.
- **Auditability.** `run_manifest.json` records tool version, profile, config hash, target SHA, per-stage cost, and quality metrics.

Roadmap (not yet implemented): Jira historical-context loader, Databricks feedback backend, update-same-comment for multi-commit PRs, SIEM log aggregation, full enterprise CMDB/CVE enrichment wiring.

8. Limitations (honest framing)

- **LLM-generated, non-deterministic.** Findings are triage candidates, not confirmed vulns; two runs can differ unless every backend is mock.
- **Token-hungry.** Caps are per-stage — use `estimate` and the DoS limits.
- **Elevated privilege.** Run only against trusted repos by authorized operators.

- **Feedback loop is local-first.** SQLite at `~/redeye/scans.db`; a Databricks backend is on the roadmap.
 - **Comparison framing.** Improvement claims are *architectural* (more/earlier deterministic filters, more backends, CI/feedback) rather than benchmarked percentages; quantify them on your own corpus with the `hallucination_metrics` the harness emits.
-

9. References

- In-repo: `README.md`, `docs/architecture.md`, `docs/REDUCING_HALLUCINATIONS.md`, `docs/configuration.md`, `docs/CI_AND_FEEDBACK.md`, `CHANGELOG.md`
- Source: `redeye/structural.py`, `redeye/grounding.py`, `redeye/scope.py`, `redeye/cache.py`, `redeye/baseline.py`, `redeye/schema.py`
- Upstream: Visa Vulnerability Agentic Harness (VVAH), Apache-2.0 — see NOTICE